CYBERVERSE

Technology Stack — What's Built vs. What's Planned

This document explains, honestly, what technology actually powers the hackathon build you can play right now, and what's on the roadmap for a production version. We've kept these clearly separated so there's no ambiguity about what's real today.

1. What Powers the Hackathon Build (Live Today)

The playable version was deliberately built as a single, self-contained HTML file with zero external build tooling — this was a conscious engineering choice, not a shortcut. It means the game runs by opening one file in any browser, with no npm install, no server, and no risk of a broken CDN link during a live demo.

Layer	Technology	Why this choice
UI framework	React 18	Component-driven architecture keeps the game's many screens (boot, dashboard, mission, debrief) organized and reusable.
JSX compilation	Babel Standalone (in-browser)	Transforms JSX to plain JavaScript directly in the browser at load time, so no build step or bundler is needed to run the game.
Audio engine	Tone.js	Generates the ambient soundtrack live using synthesizers rather than audio files — the score can react instantly to game state (danger, mood, chapter).
Icon system	Hand-built inline SVG components	A custom icon renderer avoids any external icon library dependency, keeping the file fully self-contained and removing a class of loading failures.
Animation	CSS keyframes + inline transforms	All motion (glitch effects, phone tilt, particle drift, iris transitions) runs on native CSS/Canvas — no animation library overhead.
State management	React hooks (useState / useRef / useEffect)	The game's scope doesn't yet need a dedicated state library — hooks keep player stats, mission progress, and UI state predictable.

Systems built specifically for this game

- **Mood-reactive music engine** — the ambient score transposes into a different key and filter brightness for each chapter's accent color, so a phishing-email case and a digital-arrest case are sonically distinct, not just visually.
- **Success/failure audio cues** — a short generated chime plays instantly based on whether a choice was correct, synthesized live rather than played from an audio file.
- **Investigation engine** — a reusable component (chat / call / email / QR) that renders any scam type from a single data structure, so adding a new case is a content change, not a code change.

• **Stress-linked countdown clock** — the timer's starting duration is computed from the player's current Stress stat, then rendered as a live radial clock next to the phone.

2. Planned Production Stack (Roadmap)

The single-file architecture is ideal for a hackathon demo — instant, dependency-free, impossible to break on a venue's wifi. Scaling to a real, ongoing product means introducing a proper backend, persistence, and content generation pipeline. None of this is built yet — it's the honest next step.

Layer	Planned technology	Purpose
Build tooling	Vite + React (multi-file)	Proper module bundling, hot-reload development, and optimized production builds.
State management	Zustand	Lightweight global state for player progress, stats, and inventory as the app grows beyond a few screens.
Backend & auth	Firebase (Auth + Firestore)	User accounts, cross-device save/progress sync, and cached mission storage.
AI content generation	Gemini API (server-side only)	Generates new mission scenarios, scam artifacts, and mentor explanations so no two playthroughs repeat — called only from a backend function, never from the browser, so no API key is ever exposed.
Content moderation	Server-side filtering layer	Every AI-generated scam artifact is checked before reaching a player, so nothing generated could function as a real usable phishing template.
Distribution	Progressive Web App (PWA)	Installable on a phone home screen and playable offline for static content — important for reaching lower-connectivity areas of India.

3. Why This Split Matters

A hackathon project that only exists as a slide deck promise is easy to dismiss. A hackathon project that is genuinely playable, end to end, on real infrastructure choices — even a deliberately simple one — demonstrates that the team can ship, not just pitch. The migration path from the current single-file build to the planned stack is a scaling exercise, not a rebuild: the game logic, data structures, and component design already map directly onto Zustand stores and Firestore collections.
